

DSA & Advanced DSA Roadmap

Phase 4 — Stack

Learning Objectives

By the end of this phase, learners will be able to:

- Explain the LIFO (Last-In-First-Out) principle and implement a stack using both an array and a linked list.
- Connect the abstract stack data structure to the function call stack and recursion.
- Identify and apply core stack patterns: balanced parentheses checking, monotonic stacks, next-greater-element queries, histogram-style area problems, and expression evaluation.
- Solve Easy to Medium level stack problems independently, and approach Hard histogram/expression problems with a guided monotonic-stack technique.

Implementation & Internals

Core Concepts

- Array Stack — implementing push/pop/peek backed by a dynamic array, and why operations are $O(1)$ amortized.
- Linked List Stack — implementing push/pop/peek backed by a singly linked list with head insertion.
- Function Call Stack — how a program's runtime stack stores return addresses and local variables for each function call.
- Recursion Stack — why deep recursion can cause a stack overflow, and how this connects back to Phase 0's recursion fundamentals.

Implementation Exercises

- Build a stack from scratch using a dynamic array, supporting push, pop, peek, and isEmpty.
- Build an equivalent stack from scratch using a singly linked list, and compare its trade-offs against the array version.
- Trace the call stack by hand for a small recursive function, drawing each frame as it is pushed and popped.
- Deliberately write a function with no base case and observe (in a sandboxed environment) the stack overflow it causes.

Pattern Mastery

Pattern 1: Balanced Parentheses

Concepts

- Using a stack to match opening and closing brackets in order.
- Pushing opening brackets, popping and comparing on closing brackets.
- Extending the matching idea to counting/inserting the minimum number of brackets needed for validity.

Practice Problems

- Check whether a string of brackets is valid.
- Find the minimum number of insertions needed to make a parentheses string valid.
- Remove the minimum number of invalid parentheses from a string containing letters and brackets.
- Generate all strings reachable by removing the minimum number of invalid parentheses.

LeetCode

- [Valid Parentheses — leetcode.com/problems/valid-parentheses](https://leetcode.com/problems/valid-parentheses)
- [Minimum Add to Make Parentheses Valid — leetcode.com/problems/minimum-add-to-make-parentheses-valid](https://leetcode.com/problems/minimum-add-to-make-parentheses-valid)
- [Minimum Remove to Make Valid Parentheses — leetcode.com/problems/minimum-remove-to-make-valid-parentheses](https://leetcode.com/problems/minimum-remove-to-make-valid-parentheses)
- [Remove Invalid Parentheses — leetcode.com/problems/remove-invalid-parentheses](https://leetcode.com/problems/remove-invalid-parentheses)

Pattern 2: Monotonic Stack

Concepts

- Maintaining a stack whose elements are always increasing or always decreasing.
- Popping elements that violate the monotonic property before pushing the new element.
- Why this guarantees each element is pushed and popped at most once, giving $O(n)$ total time.

Practice Problems

- For each day, find how many days until a warmer temperature.
- Find the next greater element for each element in an array, including the circular variant.

LeetCode

- [Daily Temperatures — leetcode.com/problems/daily-temperatures](https://leetcode.com/problems/daily-temperatures)
- [Next Greater Element II — leetcode.com/problems/next-greater-element-ii](https://leetcode.com/problems/next-greater-element-ii)

Pattern 3: Next Greater Element

Concepts

- Finding, for each element, the first element to its right (or in a circular sense) that is strictly greater.
- Mapping next-greater relationships from one array into queries on a related array using a hash map.

Practice Problems

- Given two arrays where one is a subset of the other, find the next greater element for each query value.

LeetCode

- [Next Greater Element I — leetcode.com/problems/next-greater-element-i](https://leetcode.com/problems/next-greater-element-i)

Note: Next Greater Element II (circular array) is listed under Monotonic Stack above, since it is the same core technique applied circularly. It is not repeated here.

Pattern 4: Histogram Problems

Concepts

- Treating each bar as a potential limiting height and finding how far it can extend left and right.
- Using a monotonic increasing stack of indices to find, for each bar, the nearest shorter bar on each side in $O(n)$ total time.
- Computing area as height $\times$ width once both boundaries are known.

Practice Problems

- Find the area of the largest rectangle that can be formed in a histogram.

LeetCode

- [Largest Rectangle in Histogram — leetcode.com/problems/largest-rectangle-in-histogram](https://leetcode.com/problems/largest-rectangle-in-histogram)

Note: this is a Hard problem placed here deliberately as the canonical histogram-pattern exercise; it appears again under the Phase Assessment's Hard tier rather than as a separate, different problem.

Pattern 5: Expression Evaluation

Concepts

- Using a stack to evaluate postfix (Reverse Polish) expressions directly, without needing operator precedence rules.
- Designing a stack-backed data structure that also supports an auxiliary operation (like retrieving the minimum) in $O(1)$.
- Simulating a sequence of stack-like operations described as a list of commands.

Practice Problems

- Evaluate an arithmetic expression given in Reverse Polish Notation.
- Design a stack that supports push, pop, top, and retrieving the minimum element, all in $O(1)$.
- Simulate the scoring rules of a simplified baseball game using a stack of round scores.

LeetCode

- [Evaluate Reverse Polish Notation — leetcode.com/problems/evaluate-reverse-polish-notation](https://leetcode.com/problems/evaluate-reverse-polish-notation)
- [Min Stack — leetcode.com/problems/min-stack](https://leetcode.com/problems/min-stack)
- [Baseball Game — leetcode.com/problems/baseball-game](https://leetcode.com/problems/baseball-game)

Phase Assessment

Implementation Tasks

- Build a stack from scratch using a dynamic array, and a second version using a linked list.
- Implement a function that uses your own stack (not a library stack) to check balanced parentheses.
- Analyze and explain why a monotonic stack gives $O(n)$ total time despite the nested-looking while loop inside the for loop.

Recommended LeetCode Target

Easy Problems

- leetcode.com/problems/valid-parentheses
- leetcode.com/problems/baseball-game
- leetcode.com/problems/minimum-add-to-make-parentheses-valid
- leetcode.com/problems/next-greater-element-i

Medium Problems

Balanced Parentheses

- leetcode.com/problems/minimum-remove-to-make-valid-parentheses

Monotonic Stack

- leetcode.com/problems/daily-temperatures
- leetcode.com/problems/next-greater-element-ii

Expression Evaluation

- leetcode.com/problems/min-stack
- leetcode.com/problems/evaluate-reverse-polish-notation

Hard Problems

- leetcode.com/problems/largest-rectangle-in-histogram
- leetcode.com/problems/remove-invalid-parentheses

On scope and dedup

This phase has 11 distinct problems across 4 difficulty groupings — more than the 5–7 problems typical of Phases 1–3, per your instruction to expand the target list. Each one maps to exactly one pattern above (no problem is double-counted across patterns), and none overlaps with any problem used in Phases 0–3. The full cross-phase ledger is maintained separately and updated after every phase.

Coding Challenges (Assessment Day)

- Valid Parentheses
- Daily Temperatures
- Evaluate Reverse Polish Notation
- Min Stack (design)
- Largest Rectangle in Histogram

Expected Outcome

Students should be able to:

- Implement a stack from scratch using both an array and a linked list, and explain the trade-offs between them.
- Recognize common stack patterns: balanced parentheses, monotonic stacks, next-greater-element queries, histogram-style area problems, and expression evaluation.
- Solve unseen stack problems using pattern-based thinking rather than memorized solutions.
- Complete Easy and Medium stack questions independently, and approach Hard problems (histogram area, invalid parentheses removal) using the monotonic-stack and backtracking techniques introduced this phase.

Once this phase is approved, Phase 5 (Queue) will follow, continuing the same phase-by-phase, document-by-document delivery with the expanded problem-count standard and strict cross-phase dedup applied going forward.